

Resolved-Rate Motion Control (Lab:2)

Pravin Oli (u1999699), IFROS

Abstract—This lab includes kinematic simulations and the implementation of three control strategies: Transpose, Pseudoinverse, and Damped Least Squares (DLS). The kinematic model is derived using Denavit-Hartenberg (DH) parameters, and the Jacobian-based control methods are analyzed. The results demonstrate the effectiveness of each approach in guiding the end-effector towards a desired position, with a focus on error reduction and trajectory optimization. The pseudoinverse and DLS methods exhibit superior performance in terms of accuracy and stability compared to the transpose method.

1 INTRODUCTION

THE The lab consists of two exercises: Kinematic Simulation: for implementing the forward kinematics of a planar robotic manipulator and Resolved-Rate Motion Control: for Implementing the control algorithm to drive the robot towards a desired Cartesian position using three control methods: Transpose, Pseudoinverse, and Damped Least Squares (DLS).

1.1 Objectives

This lab aims to implement and analyze the kinematic simulation and control of a planar robotic manipulator using Denavit-Hartenberg (DH) parameters and Resolved-Rate Motion Control (RRMC) techniques.

1.2 Task-1

Implement forward kinematics using DH parameters to compute link transformations and update the robot's structure.

1.3 Task-2

Extend Task 1 by implementing RRMC to move the robot's end-effector to a desired position using: Transpose, Pseudoinverse, and DLS control methods.

2 PLANER ROBOTIC MANIPULATOR

A planar manipulator operates in a two-dimensional plane, typically with revolute or prismatic joints.

2.1 2D Revolute Joint Manipulator

A planar 2-link revolute manipulator consists of two rotational joints that operate in the XY-plane. The configuration of the manipulator is described by the joint angles θ_1 and θ_2 .

$$\text{State} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}$$

where: - θ_1 is the angle of the first revolute joint. - θ_2 is the angle of the second revolute joint.

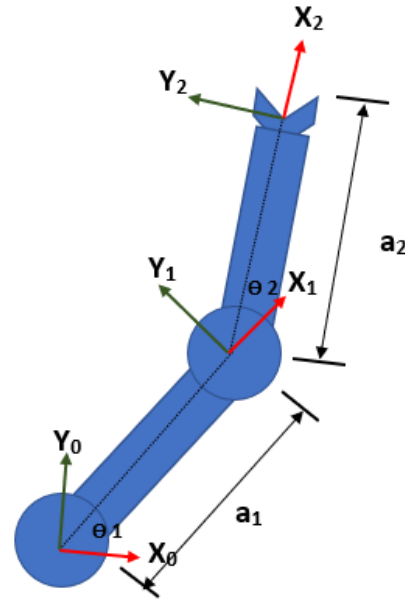


Fig. 1: 2D planar manipulator

2.2 Denavit-Hartenberg Formulation

The Denavit-Hartenberg (DH) convention provides a standardized method for describing the kinematics of robotic manipulators by defining the transformation between consecutive links. The transformation matrix for each link is given by:

$$T_n^{n-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & 0 \\ \sin \theta_i & \cos \theta_i & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha_i & -\sin \alpha_i & 0 \\ 0 & \sin \alpha_i & \cos \alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The total transformation from the base to the end-effector is obtained by multiplying the individual transformations:

$$T_n^0 = T_1^0 T_2^1 \dots T_n^{n-1}$$

- Erasmus Mundus Joint Masters In Intelligent Field Robotic Systems, University of Girona, Spain. E-mail: pravin.oli.08@gmail.com.

$$T_n^{n-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where: - θ_i is the joint angle (rotation about the z -axis), - d_i is the link offset (translation along the z -axis), - a_i is the link length (translation along the x -axis), - α_i is the link twist (rotation about the x -axis).

2.3 Jacobian Computation

The Jacobian matrix represents the relationship between joint velocities and end-effector velocities in a robotic manipulator.

For a multi-link manipulator, the Jacobian is computed for each joint in the chain. The full Jacobian matrix is constructed as:

$$J_n^0 = [J_1 \quad J_2 \quad J_3 \quad \dots \quad J_n]$$

where each column J_i depends on whether the joint is revolute or prismatic:

Revolute joint:

$$J_i = \begin{bmatrix} z_{i-1} \times (O_n - O_{i-1}) \\ z_{i-1} \end{bmatrix}$$

2.4 Resolved-Rate Motion Control

2.4.1 Transpose Method

The Jacobian Transpose Method is a simple approach that avoids explicit inversion. The joint velocities are computed as:

$$\zeta = J^T \dot{x}_E$$

where: - J^T is the **transpose of the Jacobian matrix**, - $\dot{x}_E$ is the desired end-effector velocity. This method is easy to implement and works well when the Jacobian is well-conditioned.

2.4.2 Pseudoinverse Method

The Pseudoinverse is used for solving inverse kinematics when the Jacobian is square/non-square and full-rank. The joint velocities are given by:

$$\zeta = J^+ \dot{x}_E$$

where: - $J^+ = J^T(JJ^T)^{-1}$ is the pseudoinverse of the Jacobian matrix. - $\dot{x}_E$ is the desired end-effector velocity. This method is useful for redundant manipulators and inverse kinematics problems.

2.4.3 Damped Least Squares (DLS) Method

To avoid numerical instability near singularities, the **Damped Least Squares (DLS) Method** modifies the inverse computation by introducing a damping factor λ :

$$\zeta = J^T(JJ^T + \lambda^2 I)^{-1} \dot{x}_E$$

where: - λ is the **damping factor**, ensuring numerical stability. - I is the **identity matrix**. - This method works for **square/non-square Jacobians**, even if they are rank-deficient. The DLS method improves robustness and avoids large joint velocities near singularities.

3 IMPLEMENTATION

This lab is implemented in three python scripts: Common.py, Exercise 1.py and Exercise 2.py

3.1 Common.py

Below is an implementation of Common.py:

```
1 import numpy as np
2
3 # Denavit-Hartenberg (DH) Transformation Matrix
4 def DH(d, theta, a, alpha):
5     # 1. Build matrices representing elementary
6     #    transformations (based on input parameters).
7     T_d=np.array([[1,0,0,0],
8                  [0,1,0,0],
9                  [0,0,1,d],
10                 [0,0,0,1]])
11
12     T_theta=np.array([[np.cos(theta),-np.sin(theta),
13                      ,0,0],
14                      [np.sin(theta),np.cos(theta),
15                      ,0,0],
16                      [0,0,1,0],
17                      [0,0,0,1]])
18
19     T_a=np.array([[1,0,0,a],
20                  [0,1,0,0],
21                  [0,0,1,0],
22                  [0,0,0,1]])
23
24     T_alpha=np.array([[1,0,0,0],
25                      [0,np.cos(alpha),-np.sin(alpha),
26                      ,0],
27                      [0,np.sin(alpha),np.cos(alpha),
28                      ,0],
29                      [0,0,0,1]])
30
31     # 2. Multiply matrices in the correct order (
32     #    result in T).
33     T=T_d @ T_theta @ T_a @ T_alpha
34
35     return T
36
37 # Forward Kinematics Computation
38 def kinematics(d, theta, a, alpha):
39     # Initialize the base transformation matrix (
40     #    Identity matrix)
41     T = [np.eye(4)]
42
43     # For each set of DH parameters:
44     # 1. Compute the DH transformation matrix.
45     # 2. Compute the resulting accumulated
46     #    transformation from the base frame.
47     # 3. Append the computed transformation to T.
48
49     for i in range(len(d)):
50         Ti = DH(d[i], theta[i], a[i], alpha[i]) #
51         # Compute DH transformation
52         T.append(T[-1] @ Ti) # Compute cumulative
53         # transformation
54     return T
55
56 # Compute Jacobian Matrix, Inverse kinematics
57 def jacobian(T, revolute):
58     # 1. Initialize J and 0.
59     # 2. For each joint of the robot
60     #    a. Extract z and o.
61     #    b. Check joint type.
62     #    c. Modify corresponding column of J.
63
64     # 1. Initialize J and 0.
65     J = np.zeros((6,len(T)-1))
66
67     # 2. For each joint of the robot
68     for i in range (len(T)-1):
69         # a. Extract z and o.
70         if i == 0:
71             z = np.array([0,0,1]) #Base Z-axis
```

```

66     else:
67         z = T[i][0:3,2] # Extract the Z-axis
                           from the transformation matrix
68
69     o = T[-1][0:3, 3] - T[i][0:3, 3] # Compute
                           position vector
70
71     zero = np.zeros(3)
72
73     # b. Check joint type.
74     if revolute[i]:
75
76     # c. Modify corresponding column of J.
77         J[:,i] = np.concatenate((np.cross(z, o),
78                                   z)).reshape((6,))
79     else:
80         J[:,i] = np.concatenate((z,zero)).
81         reshape((6,))
82     return J
83
84 # Damped Least-Squares Inverse of Jacobian
85 def DLS(J, damping):
86     # Zeta = J(      ).T (J(      )*J(      ).T + **2 *
87         I) 1      xE
88     DLS = J.T @ np.linalg.inv(J @ J.T + damping**2 *
89                                np.eye(J.shape[0]))
90     return DLS
91
92 # Extract characteristic points of a robot projected
93     on X-Y plane
94 def robotPoints2D(T):
95     P = np.zeros((2,len(T))) # Create an empty array
96     for 2D coordinates
97
98     for i in range(len(T)):
99         P[:,i] = T[i][0:2,3] # Extract X and Y
100         positions
101     return P
102
103 path, = ax.plot([], [], 'r-', lw=1) # End-effector
104     path
105
106 # Memory
107 PPx = []
108 PPy = []
109 q1 = []
110 q2 = []
111 timestamp = []
112
113 # Simulation initialization
114 def init():
115     line.set_data([], [])
116     path.set_data([], [])
117     return line, path
118
119 # Simulation loop
120 def simulate(t):
121     global d, q, a, alpha
122     global PPx, PPy
123
124     # Update robot
125     T = kinematics(d, q, a, alpha)
126     dq = np.array([0.3,0.9]) # Define how joint
127         velocity changes with time!
128     q = q + dt * dq
129
130     # Update drawing
131     PP = robotPoints2D(T)
132     line.set_data(PP[0,:], PP[1,:])
133     PPx.append(PP[0,-1])
134     PPy.append(PP[1,-1])
135     path.set_data(PPx, PPy)
136
137     # Update memory for plotting
138     q1.append(q[0])
139     q2.append(q[1])
140     timestamp.append(t)
141
142     return line, path
143
144 # Run simulation
145 animation = anim.FuncAnimation(fig, simulate, tt,
146                                interval=10, blit=
147                                True, init_func=
148                                init, repeat=
149                                False)
150
151 plt.show()
152
153 # after animation time vs angle plot
154 fig_joint = plt.figure()
155 ax = fig_joint.add_subplot()
156 ax.set_title('joint positions')
157 ax.set_xlabel('Time[s]')
158 ax.set_ylabel('Angle[rad]')
159 ax.set_aspect('equal')
160 ax.grid(True)
161 plt.plot(timestamp,q1,label='q1')
162 plt.plot(timestamp,q2,label='q2')
163 plt.legend()
164 plt.show()

```

3.2 Exercise 1.py

Below is an implementation of Exercise 1.py:

```

1 # Import necessary libraries
2 from Common import * # Import our library (includes
   Numpy)
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as anim
5
6 # Robot definition (planar 2 link manipulator)
7 d = np.zeros(2) # displacement along Z-
   axis
8 q = np.array([0.2, 0.5]) # rotation around Z-axis (
   theta)
9 a = np.array([0.75, 0.5]) # displacement along X-
   axis
10 alpha = np.zeros(2) # rotation around X-axis
11
12 # Simulation params
13 dt = 0.01 # Sampling time
14 Tt = 20 # Total simulation time
15 tt = np.arange(0, Tt, dt) # Simulation time vector
16
17 # Drawing preparation
18 fig = plt.figure()
19 ax = fig.add_subplot(111, autoscale_on=False, xlim
   =(-2, 2), ylim=(-2,2))
20 ax.set_title('Kinematics')
21 ax.set_xlabel('x[m]')
22 ax.set_ylabel('y[m]')
23 ax.set_aspect('equal')
24 ax.grid()
25 line, = ax.plot([], [], 'o-', lw=2) # Robot
   structure
26
27 # Import necessary libraries
28 from Common import * # Includes numpy import
29 import numpy as np
30 import matplotlib.pyplot as plt
31 import matplotlib.animation as anim
32
33 # Robot definition
34 d = np.zeros(2) # Displacement along Z-
   axis
35
36 # after animation time vs angle plot
37 fig_joint = plt.figure()
38 ax = fig_joint.add_subplot()
39 ax.set_title('joint positions')
40 ax.set_xlabel('Time[s]')
41 ax.set_ylabel('Angle[rad]')
42 ax.set_aspect('equal')
43 ax.grid(True)
44 plt.plot(timestamp,q1,label='q1')
45 plt.plot(timestamp,q2,label='q2')
46 plt.legend()
47 plt.show()

```

3.3 Exercise 2.py

Below is an implementation of Exercise 2.py:

```

1 # Import necessary libraries
2 from Common import * # Includes numpy import
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import matplotlib.animation as anim
6
7 # Robot definition
8 d = np.zeros(2) # Displacement along Z-
   axis
9
10 # after animation time vs angle plot
11 fig_joint = plt.figure()
12 ax = fig_joint.add_subplot()
13 ax.set_title('joint positions')
14 ax.set_xlabel('Time[s]')
15 ax.set_ylabel('Angle[rad]')
16 ax.set_aspect('equal')
17 ax.grid(True)
18 plt.plot(timestamp,q1,label='q1')
19 plt.plot(timestamp,q2,label='q2')
20 plt.legend()
21 plt.show()

```

```

9 q_init = np.array([0.2, 0.5]) # Initial joint
  angles
10 a = np.array([0.75, 0.5]) # Displacement along X-
  axis
11 alpha = np.zeros(2) # Rotation around X-axis
12 revolute = [True, True]
13 sigma_d = np.array([0.0, 1.0]) # Desired end-
  effector position
14 K = np.diag([1, 1]) # Control gain
15
16 # Simulation parameters
17 dt = 1.0 / 60.0
18
19 # Control methods to test
20 controllers = ["transpose", "pseudoinverse", "DLS"]
21
22 # Storage for error norms
23 error_storage = {ctrl: [] for ctrl in controllers}
24
25 # Define colors to plot path for different methods
26 controller_colors = {
27     "transpose": 'b', # Blue
28     "pseudoinverse": 'orange', # Orange
29     "DLS": 'g', # Green
30 }
31
32 # Drawing preparation
33 figs = {}
34 axes = {}
35 lines = {}
36 paths = {}
37 points = {}
38 PPxs = {}
39 PPys = {}
40 qs = {} # Store independent q values for each
  controller
41
42 for ctrl in controllers:
43     figs[ctrl] = plt.figure()
44     axes[ctrl] = figs[ctrl].add_subplot(111,
45         autoscale_on=False, xlim=(-2, 2), ylim=(-2,
46         2))
47     axes[ctrl].set_title(f'Simulation - {ctrl}')
48     axes[ctrl].set_aspect('equal')
49     axes[ctrl].grid()
50     lines[ctrl], = axes[ctrl].plot([], [], 'o-', lw
51     =2)
52     paths[ctrl], = axes[ctrl].plot([], [],
53     controller_colors[ctrl], lw=1)
54     points[ctrl], = axes[ctrl].plot([], [], 'rx')
55     PPxs[ctrl] = []
56     PPys[ctrl] = []
57     qs[ctrl] = np.array(q_init) # Initialize
58     independent q values for each controller
59
60 # Simulation initialization
61 def init():
62     for ctrl in controllers:
63         lines[ctrl].set_data([], [])
64         paths[ctrl].set_data([], [])
65         points[ctrl].set_data([], [])
66     return sum([[lines[ctrl], paths[ctrl], points[
67     ctrl]] for ctrl in controllers], [])
68
69 # Controller function
70 def controller(type, J):
71     if type == "transpose":
72         return J.T
73     elif type == "pseudoinverse":
74         return np.linalg.pinv(J)
75     elif type == "DLS":
76         return DLS(J, 0.1) # Assuming DLS is
77         defined elsewhere
78     else:
79         raise ValueError("Unknown controller method
80         ")
81
82 # Simulation loop (stores errors)
83 def simulate(t, current_controller):
84     global d, a, alpha, revolute, sigma_d,
85     error_storage
86
87     q = qs[current_controller] # independent q
88     value per controller
89
90     # Update robot
91     T = kinematics(d, q, a, alpha)
92     J = jacobian(T, revolute)[0:2, :] # Extract
93     first two rows for planar case
94     P = robotPoints2D(T) # Extract points
95
96     # Compute error
97     sigma = np.array([P[0, -1], P[1, -1]]) #
98     Position of the end-effector
99     err = sigma_d - sigma # Control error
100     dq = controller(current_controller, J) @ (K @
101     err) # Compute control solution
102     q += dt * dq # Integrate to update joint angles
103     qs[current_controller] = q # Store updated q
104
105     # Store error norm
106     error_storage[current_controller].append(np.
107     linalg.norm(err))
108
109     # Update drawing
110     lines[current_controller].set_data(P[0, :], P[1,
111     :])
112     PPxs[current_controller].append(P[0, -1])
113     PPys[current_controller].append(P[1, -1])
114     paths[current_controller].set_data(PPxs[
115     current_controller], PPys[current_controller]
116     ])
117     points[current_controller].set_data(sigma_d[0],
118     sigma_d[1])
119
120     return lines[current_controller], paths[
121     current_controller], points[
122     current_controller]
123
124 # Run simulations for all controllers
125 animations = []
126 for ctrl in controllers:
127     animation = anim.FuncAnimation(figs[ctrl],
128     simulate, np.arange(0, 10, dt),
129     fargs=(ctrl,),
130     interval=10,
131     blit=True,
132     init_func=
133     init, repeat=
134     False)
135     animations.append(animation)
136
137 # Show animation plots
138 plt.show()
139
140 # **Plot Control Error Norms After Simulation
141 # Completes**
142 fig_error, ax_error = plt.subplots()
143 ax_error.set_title('Control Error Norm Evolution')
144 ax_error.set_xlabel('Time Steps')
145 ax_error.set_ylabel('Error Norm')
146 ax_error.grid(True)
147
148 for ctrl in controllers:
149     ax_error.plot(error_storage[ctrl], label=ctrl,
150     color=controller_colors[ctrl], lw=1.0)
151
152 ax_error.legend()
153 plt.show()
154
155 # Plot Path Comparison of path for All Controllers
156 fig_path, ax_path = plt.subplots()
157 ax_path.set_title('Path Comparison of Controllers')
158 ax_path.set_xlabel('X Position')

```

```

131 ax_path.set_ylabel('Y Position')
132 ax_path.grid(True)
133
134 # Plot the recorded paths for all controllers
135 for ctrl in controllers:
136     ax_path.plot(PPxs[ctrl], PPys[ctrl], label=ctrl,
137                 color=controller_colors[ctrl], lw=2)
138 ax_path.legend()
139 plt.show()

```

4 RESULTS AND DISCUSSION

The result of lab is obtained as:

4.1 Kinematic Simulation

The kinematics plot illustrates the movement of the 2-link manipulator in a Cartesian plane, robot's structure, and trajectory of the end-effector. The transformation matrix, computed using $T = \text{kinematics}(d, q, a, \alpha)$, continuously updates the robot's configuration, influencing the end-effector's motion. As the joint angles change, the robot moves dynamically, tracing a looping trajectory in space, showcasing the path it follows during the simulation.

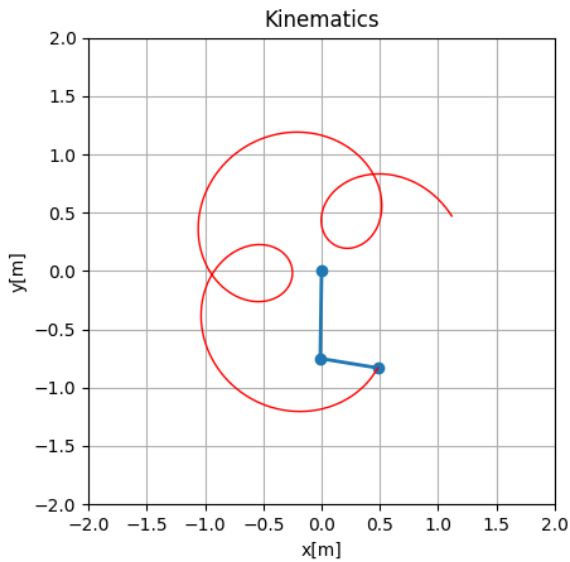


Fig. 2: Kinematic Simulation

The joint position plot represents the time evolution of the two joint angles, q_1 and q_2 , of a 2-link manipulator. Throughout the simulation, the joint angles are updated based on a predefined velocity, with values being stored in q_1 and q_2 lists. The plot is generated using `plt.plot(timestamp, q1, label='q1')` and `plt.plot(timestamp, q2, label='q2')`, showing a linear increase in both angles over time. This indicates that the joints are moving at a constant rate, as defined by `[dq = np.array([0.3, 0.9]).]`

4.2 Control Method Simulation

4.2.1 Transpose Method

The transpose method uses the Jacobian transpose for control. The simulation shows that while it successfully moves the end-effector to the target, the trajectory is relatively curved and takes longer compared to other methods. This is expected due to the lack of optimality in the Jacobian transpose approach.

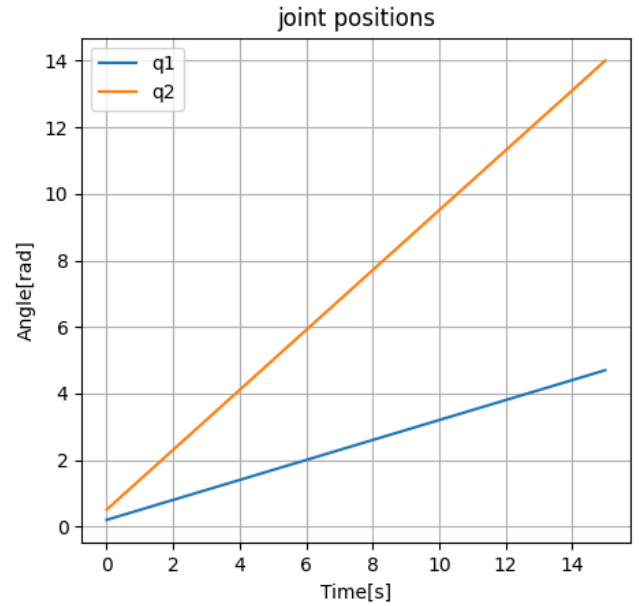


Fig. 3: joint position plot

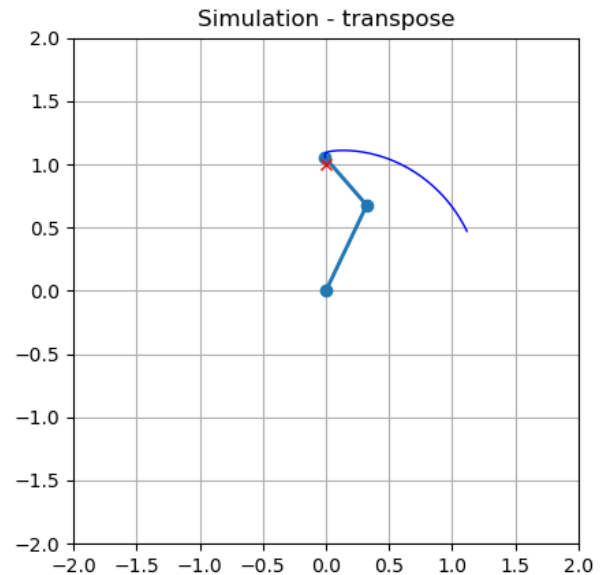


Fig. 4: Transpose Method

4.2.2 Pseudoinverse Simulation

The pseudoinverse method computes the Moore-Penrose inverse of the Jacobian to solve for joint velocities. This method results in a more direct and optimal path toward the target compared to the transpose method. The control accuracy is also improved.

4.2.3 DLS Simulation

The DLS method incorporates a damping factor to improve numerical stability. The trajectory is similar to that of the pseudoinverse method but with slightly smoother motion. This method is effective in avoiding singularities and ensuring stable convergence.

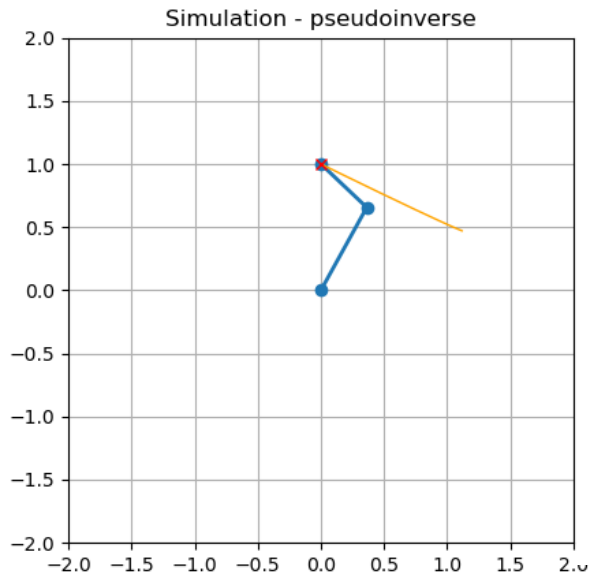


Fig. 5: PseudoInverse Simulation

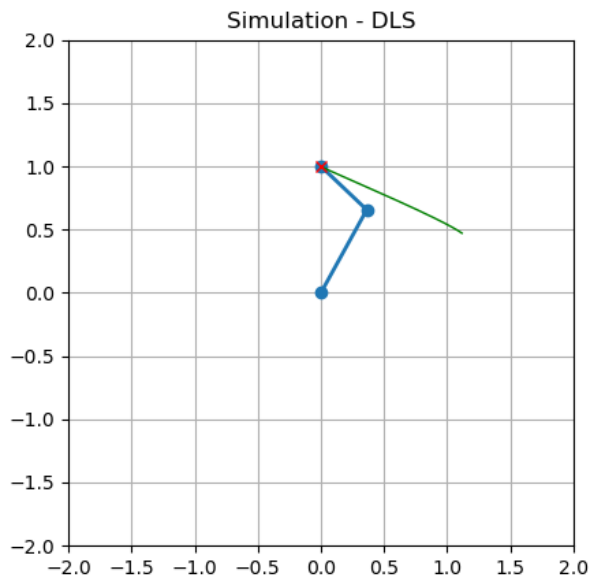


Fig. 6: DLS Simulation

4.2.4 Evolution of the control error

The evolution of the control error norm for the three methods shows that the pseudoinverse and DLS methods exhibit faster error convergence compared to the transpose method, confirming their superior efficiency in reducing the end-effector's error over time.

4.2.5 Path comparison of controller

The comparison of the paths taken by the end-effector using different controllers reveals that the pseudoinverse and DLS methods result in nearly straight-line paths, demonstrating their efficiency, whereas the transpose method follows a significantly curved trajectory.

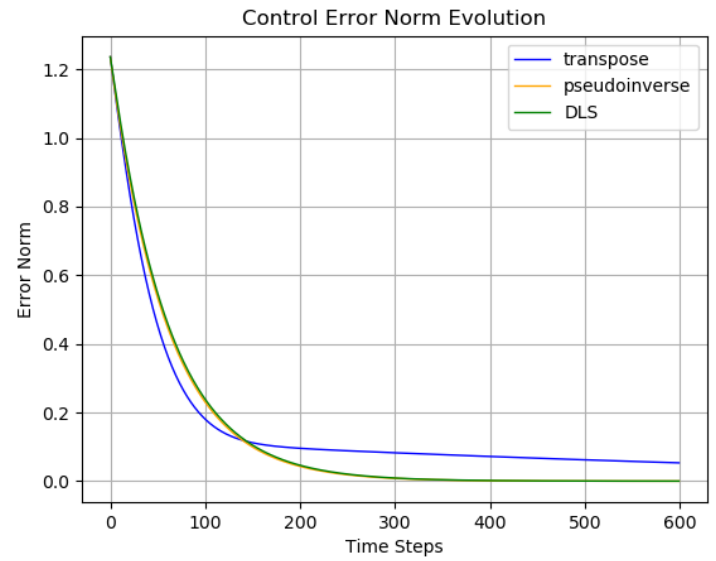


Fig. 7: Evolution of the control error

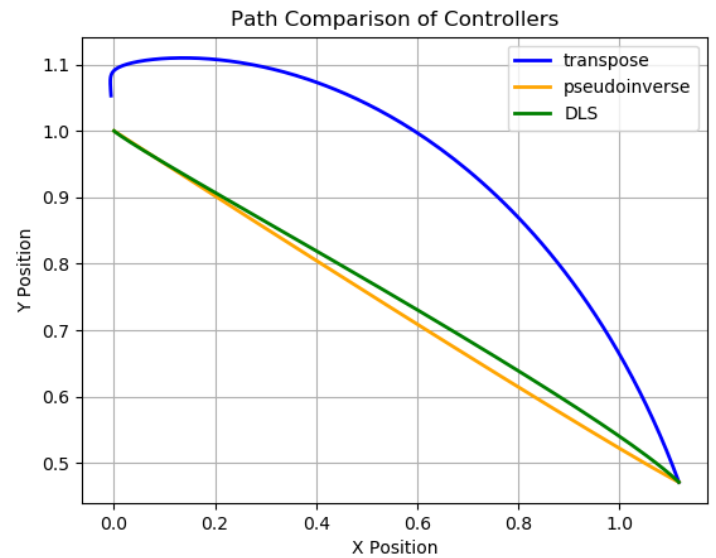


Fig. 8: Path comparison of controller

5 QUESTIONS

1. What are the advantages and disadvantages of using kinematic control in robotic systems?

Advantages:

- Computationally efficient as it does not require dynamic modeling.
- Suitable for precise motion planning in structured environments.
- Easier to implement for applications where forces and torques are not critical.

Disadvantages:

- Ignores dynamics (inertia, forces, and torques), leading to poor performance in high-speed or force-sensitive tasks.
- May result in infeasible motions if dynamic constraints are not considered.

- Less effective in unstructured environments where interactions with external forces occur.
2. Give examples of control algorithms that may be used in the robot's hardware to follow the desired velocities of the robot's joints, being the output of the resolved-rate motion control algorithm.
- **PID (Proportional-Integral-Derivative) Control**
 - **Feedforward Control**
 - **Computed Torque Control**
 - **Sliding Mode Control**
 - **Adaptive Control**
 - **Model Predictive Control (MPC)**

6 CONCLUSION

The results of this lab highlight the effectiveness of different control strategies for Resolved-Rate Motion Control. The transpose method, though simple, results in longer and less optimal trajectories. The pseudoinverse approach improves accuracy and efficiency by directly computing an optimal joint velocity solution. The DLS method further enhances stability and robustness, particularly in avoiding singularities. The error norm evolution confirms that the pseudoinverse and DLS methods converge faster to the desired position